Answer Set Programs with Intensional Functions and Aggregates

Anonymous submission

Abstract

We introduce a new framework for Answer Set Programming (ASP) with intensional functions and aggregates that generalized several existing approaches. We use this framework to define an extension of ASP where user can define their own functions, implement a new system called `funasp` that extends `clingo` to support this new language, and show that the increase in expressivity does not come with a performance penalty.

1 Introduction

Answer set programming (ASP) is a declarative programming paradigm well-suited for solving knowledge-intensive search and optimization problems (Lifschitz 2019). State-of-the art ASP systems allow two types of functions: arithmetic and Herbrand. The first type is used to perform calculations over numbers, while the second type is used to create new objects from simpler ones. Neither of these types of functions allows the programmer to define the value returned by functions. *Intensional functions* are defined by the programmer using a set of rules, analogous to how predicates are defined in logic programming (Lifschitz 2012). Programmers have been using predicates to simulate functions for a long time, but this approach has several drawbacks. For example, consider a database containing information of family relationships. Such database may contain the fact $mother(ann, eve)$. Common-sense knowledge tells us that $mother/2$ is a predicate representing a function, however, this information is not explicitly represented in the database. In most cases, programmers even omit the uniqueness constraint that enforces the functional nature of $mother/2$ and let this be the result of close world assumption. Another issue with this representation is that by looking at this fact alone, it is not clear whether ann is the mother of eve or whether eve is the mother of ann. These two issues can be addressed by using an intensional function instead of a predicate. Using the language by Cabalar (2011), the fact above can be represented as an assignment[1] $mother(ann) := eve$, which clearly indicates that $mother/1$ is a function and that eve is the mother of ann, avoiding the ambiguity of the predicate representation. Intensional functions also can be nested, allowing for more concise and natural representations. For example, $mother(mother(ann))$ clearly represents the maternal grandmother of ann, while the predicate representation requires auxiliary variables to represent the same concept.

The use of intensional functions for knowledge representation requires the ability to deal with *partial functions*. Back to our family example, in any finite non-empty database some individuals must be left with no recorded mother and, thus, the value of the function $mother/1$ is undefined for those individuals. Adding more information to the database may result in previously undefined function values becoming defined. Further elaborating this example, our database may contain a rule stating that, if the mother of a living person is undefined, we need to inquire to obtain that information:

```
inquire_mother(X):-alive(X), not mother(X)=_.
```

This rule is *non-monotonic*, as adding the required information will remove the conclusion that the information needs to be inquired. In this paper, we present a *free-logic* (Bencivenga 2002) variation of Quantified Equilibrium Logic (Pearce and Valverde 2008) that generalizes several existing approaches (Lin and Wang 2008; Cabalar 2011; Balduccini 2013; Bartholomew and Lee 2019) in the sense that we can select a particular function symbol to be interpreted as either semantics by adding a corresponding axiom. The relation between intensional functions and free logic was first observed by Fariñas del Cerro, Pearce, and Valverde (2013) and Cabalar et al. (2014). We use this logic to provide a semantics for a logic programming language that extends the syntax of ASP with intensional functions and aggregates. We show that in the absence of intensional functions, our semantics coincides with the semantics of ASP solver `clingo` (Gebser et al. 2015). We also show how intensional functions can be encoded in regular ASP by *flattening* them into predicates (Naish 1991; Rouveirol 1994) and use this encoding to develop an efficient implementation of our language on top of `clingo`. We experimentally show that the increased expressiveness of does not come at the cost of performance.

[1] Assignment are used in rule heads to define intensional functions. The above assignment states that the value of the function $mother/1$ for the argument ann is eve (See Section 4).

2 Many-sorted First-order Logic with Partial Functions

Free-logics are concerned with names that do not denote, that is, where the classical axiom $\exists x\,(x = t)$ does not hold for every term t. A common example is the division operation, which is undefined when the divisor is zero, that is, the formula $\exists x\,(x = 1/0)$ does not hold. From a knowledge representation perspective, undefined functions can also represent incomplete knowledge as the function $mother\backslash 1$ in the example of the introduction. In this section, we present the syntax and semantics of a version of many-sorted Quantified Equilibrium logic with partial functions, which is a free-logic in the above sense.

Syntax. We consider a many-sorted first-order language[2] A *(many-sorted) signature* consists of symbols of three kinds—*sorts*, *function constants*, and *predicate constants*. A reflexive and transitive *subsort* relation is defined on the set of sorts. A tuple $s_1, \ldots, s_n$ $(n \geq 0)$ of *argument sorts* is assigned to every function constant and to every predicate constant; in addition, a *value sort* is assigned to every function constant. The length of the tuple of argument sorts of a function or predicate constant is called its *arity*. A function constant of arity 0 is called an *object constant*. For every sort, an infinite sequence of *object variables* of that sort is chosen.

Terms and *atomic formulas* over a signature σ are defined recursively as usual in first-order logic, but ensuring that arguments of predicate and function constants are of the appropriate sorts. *Formulas* over σ are formed from atomic formulas and the 0-place connective $\bot$ (falsity) using the binary connectives $\wedge$, $\vee$, $\rightarrow$ and the quantifiers $\forall$, $\exists$. Other connectives are treated as abbreviations: $\neg F$ stands for $F \rightarrow \bot$, $\top$ stands for $\neg\bot$, and $F \leftrightarrow G$ stands for $(F \rightarrow G) \wedge (G \rightarrow F)$. A *theory* over σ is a set Γ of sentences over σ.

Interpretations and Satisfaction. Interpretations are defined as in many-sorted first-order logic, but allow function symbols to be mapped to partial functions. This interpretation is extended to terms recursively as follows:

- $f(t_1, \ldots, t_k)^I$ is $f^I(t_1^I, \ldots, t_k^I)$ if every t_i^I is defined with $1 \leq i \leq k$ and $f^I(t_1^I, \ldots, t_k^I)$ is defined; and
- $f(t_1, \ldots, t_k)^I$ is undefined otherwise.

In the following we use boldface letters to denote tuples of terms or other expressions. When a function defined for a term or other expression is applied to a tuple, we understand it as being applied to each element of the tuple. For instance, if $\mathbf{t}$ is a tuple $t_1, \ldots, t_n$ of ground terms then $\mathbf{t}^I$ stands for the tuple $t_1^I, \ldots, t_n^I$ of values assigned to them by I.

The satisfaction relation $\models$ for formulas is recursively defined as usual in many-sorted first-order logic, but for the case of atomic formulas, where we need to take into account that some terms may be undefined as follows:

- $I \models p(t_1, \ldots, t_k)$ if t_i^I is defined for all $i \in \{1, \ldots, k\}$ and $(t_1^I, \ldots, t_k^I) \in p^I$;
- $I \models t_1 = t_2$ if t_1^I and t_2^I are defined and are the same.

As usual, we say that I is a *model* of a theory Γ if $I \models F$ for every $F \in \Gamma$.

A function constant $f\backslash k$ is *total* w.r.t I if f^I is a total function, that is, if $f^I(\mathbf{d})$ is defined for every tuple $\mathbf{d}$ of domain elements of appropriate sorts. An interpretation I is *total* if every function constant of σ is total w.r.t. I. It is easy to see that total interpretations and models coincide with interpretations and models of standard many-sorted first-order logic.

Quantified Here-and-There with Partial Functions We say that two partial functions are *compatible* if they have the same domain. Given compatible partial functions f and g, we write $f \preceq g$ if $f(\mathbf{d})$ is defined implies that $g(\mathbf{d})$ is also defined for any possible domain elements $\mathbf{d}$. We write $f \prec g$ if $f \preceq g$ and $f \neq g$. Note that $\preceq$ and $\prec$ are preorders, that is, $\preceq$ is reflexive, transitive, but not necessarily symmetric.

We assume that the set of predicate constants and the set of function constants are partitioned into two disjoint sets that we call *intensional* and *extensional*. Intensional predicate and function constants are those that are defined by the programmer using rules, while extensional predicate and function can be defined in classical style using biconditionals. In a logical programming context, extensional predicates and functions often correspond to input data or built-in predicates and functions. In the following we use $\mathcal{P}$ and $\mathcal{F}$ to denote the sets of intensional predicate and function constants, respectively. Given interpretations H and I of a signature σ, we write $H \leq^{\mathcal{PF}} I$ if

- the domains of H and I are equal for every sort;
- $p^H \subseteq p^I$ for every predicate constant p, and $f^H \preceq f^I$ for every function constant f;
- $c^H = c^I$ for every predicate or function constant c that does not belong to $\mathcal{P} \cup \mathcal{F}$.

An *HT-interpretation* of σ is a pair $\langle H, I\rangle$ where H and I are two interpretations of σ such that $H \leq^{\mathcal{PF}} I$. In terms of Kripke models, we can think of H and I as the interpretations of the two worlds, where H is the "here" world and I is the "there" world. If I assigns a value to a term, H can assign a different value or even leave it undefined. This contrast with the approach by Cabalar (2011), where H can leave it undefined but cannot assign a different value, and the approach by Bartholomew and Lee (2019), where H can assign a different value but cannot leave it undefined.

To define the satisfaction of formulas, we first need to extend the signature σ. Let σ^I be the signature obtained from σ by adding a name d_s^* as object constants of the appropriate sort for every element d of the domain $|I|^s$ and every sort s of σ. We identify I with its extension to the new signature σ^I by defining $(d_s^*)^I = d$ for all $d \in |I|^s$. The satisfaction relation $\models_{ht}$ for HT-interpretations is now defined recursively as follows:

- $\langle H, I\rangle \models_{ht} p(t_1, \ldots, t_k)$ if $I \models p(t_1, \ldots, t_k)$ and $H \models p(t_1, \ldots, t_k)$
- $\langle H, I\rangle \models_{ht} t_1 = t_2$ if $I \models t_1 = t_2$ and $H \models t_1 = t_2$
- $\langle H, I\rangle \models_{ht} F \wedge G$ if $\langle H, I\rangle \models_{ht} F$ and $\langle H, I\rangle \models_{ht} G$
- $\langle H, I\rangle \models_{ht} F \vee G$ if $\langle H, I\rangle \models_{ht} F$ or $\langle H, I\rangle \models_{ht} G$

[2] We recall here some key concepts. For a more detailed presentation, see the work by Fandinno and Lifschitz (2023, Section 2.2). For review purposes, we reproduce this in Appendix ?? in the Suplementary Material.

- $\langle H, I\rangle \models_{ht} F \to G$ if $I \models F \to G$, and either $\langle H, I\rangle \not\models_{ht} F$ or $\langle H, I\rangle \models_{ht} G$
- $\langle H, I\rangle \models_{ht} \forall X F(X)$ if $\langle H, I\rangle \models_{ht} F(d^*)$ for each $d \in |I|^s$, where *s* is the sort of X
- $\langle H, I\rangle \models_{ht} \exists X F(X)$ if $\langle H, I\rangle \models_{ht} F(d^*)$ for some $d \in |I|^s$, where *s* is the sort of X

We write $H <^{\mathcal{PF}} I$ if $H \leq^{\mathcal{PF}} I$ and $H \neq I$. We say that a model I of Γ is a *stable model* if there does not exist an interpretation $H <^{\mathcal{PF}} I$ such that $\langle H, I\rangle \models_{ht} \Gamma$.

3 Relation to Existing Approaches

Existing approaches that allow for non-Herbrand functions can be classified into three groups depending on the type of functions they allow for: *denoting functions* (Pearce and Valverde 2008; Lin and Wang 2008), *rigid partial functions* (Cabalar 2011; Balduccini 2013) and *total (non-denoting) functions* (Bartholomew and Lee 2019). We show that all these approaches can be captured in our framework by adding appropriate axioms to the language of Section 2. We consider the following axioms:

$$\forall \mathbf{X} \exists Y (f(\mathbf{X}) = Y) \tag{1}$$
$$\forall \mathbf{X} (f(\mathbf{X}) = f(\mathbf{X}) \to \exists Y (f(\mathbf{X}) = Y)) \tag{2}$$
$$\forall \mathbf{X} (f(\mathbf{X}) = f(\mathbf{X})) \tag{3}$$

where $\mathbf{X} = X_1, \dots, X_k$ is a list of distinct variables of appropriate sorts and Y is a variable of the value sort of $f\backslash k$ different from the ones in $\mathbf{X}$. We use (1) to capture denoting functions, (2) to capture rigid partial functions, and (3) to capture total functions. By DEN, RIG, and TOT we denote the sets of all axioms (1), (2), and (3) for every function constant $f\backslash k \in \mathcal{F}$, respectively. The following results show that the semantics of the approaches mentioned above can be captured by adding the corresponding axioms.

Theorem 1. *A interpretation I is a stable model of a theory Γ*

- *w.r.t. the semantics by Pearce (2006) iff I is a stable model of $\Gamma \cup DEN$;*
- *w.r.t. the semantics by Cabalar (2011) iff I is a stable model of $\Gamma \cup RIG$;*
- *w.r.t. the semantics by Bartholomew and Lee (2019) iff I is a stable model of $\Gamma \cup TOT$.*

The semantics by Lin and Wang (2008) was defined using a grounding-and-reduct style, but given the connection between the grounding-and-reduct style and QHT-stable models, it is easy to see that their semantics coincides with the one by Pearce and Valverde (2008) and, thus, also with our semantics extended with DEN. Similarly, the semantics by Balduccini (2013) was defined using a grounding-and-reduct style, and it coincides with the one by Cabalar (2011) and discussed by Bartholomew and Lee (2014). Hence, it also coincides with our semantics extended with RIG.

4 Logic Programs

Syntax. We assume a *(program) signature* with two countably infinite sets of symbols: *basic symbols* and *program variables*. We assume that the set of basic symbols contains contains two distinct subsets, *numerals* and *symbolic constants*, in addition to the symbols inf and sup. We also assume a 1-to-1 correspondence between numerals and integers; the numeral corresponding to an integer n is denoted by $\overline{n}$. *Program terms* are defined recursively as follows: (i) basic symbols and variables are program terms; (ii) if f is a symbolic constant and $\mathbf{t}$ is a (possibly empty) tuple of terms, them $f(\mathbf{t})$ is a program term; and (iii) if t_1 and t_2 are terms, then $t_1 + t_2$ is a program term. When $\mathbf{t}$ is the empty tuple, $c()$ can be simply written as c. A program term (or any other expression) is *ground* if it contains no variables. A ground term is *precomputed* it it contains no addition symbols. We assume that a total order on precomputed terms is chosen such that

- inf is its least element and sup is its greatest element,
- for any integers m and n, $\overline{m} < \overline{n}$ iff $m < n$, and
- for any integer n and any symbolic constant c, $\overline{n} < c$.

An *atom* is an expression of the form $p(\mathbf{t})$, where p is a symbolic constant and $\mathbf{t}$ is a list of program terms. A *comparison* is of the form $t \prec t'$, where t and t' are program terms and $\prec$ is one of the *comparison symbols*:

$$= \quad \neq \quad < \quad > \quad \leq \quad \geq \tag{4}$$

An *atomic formula* is either an atom or a comparison. An *aggregate element* has the form

$$t_1, \dots, t_k : A_1, \dots, A_l \tag{5}$$

where each t_i $(1 \leq i \leq k)$ is a program term and each A_i $(1 \leq i \leq l)$ is an atomic formula. We call $A_1, \dots, A_l$ the *conditions* of the aggregate element. An *aggregate atom* is an expression of the forms $\texttt{\#sum}\{E\} \prec u$ or $\texttt{\#count}\{E\} \prec u$, where E is an aggregate element, $\prec$ is one of the comparison symbols in (4), and u is a program term, called *guard*. A *literal* is either an atomic formula, an aggregate atom, or an expression of the following forms:

$$not\ (A_1, \dots, A_l) \qquad \text{or} \qquad not\ not\ A_1$$

where each A_i $(1 \leq i \leq l)$ is an atomic formula.[3] When $m = 1$, we often write $not\ A_1$ instead of $not\ (A_1)$. Literals that are not aggregate atoms are called *basic literals*.

An *assignment* is an expression of the form

$$f(\mathbf{t})\ \texttt{:=}\ t' \tag{6}$$

where f is symbolic constant, $\mathbf{t}$ is tuple of program terms and t' is a term. In an assignment (6), we say that f is the *assigned function symbol*. A *rule* is an expression of the form

$$Hd\ \texttt{:-}\ B_1, \dots, B_n, \tag{7}$$

where Hd is an atom, an assignment or the symbol $\bot$, and each B_i $(1 \leq i \leq n)$ is a literal.

We call the symbol :- the *rule operator*. We call the left-hand side of the rule operator the *head*, the right-hand side of the rule operator the *body*. When the head of the rule is an atom we call the rule *normal*, when it is the symbol $\bot$ we call it a *constraint*, and when it is an assignment we call it an *assignment rule*. When the body of a normal rule is empty, we call the rule a *fact*. In constraints, the symbol $\bot$ is usually omitted. A *program* is a set of rules. A *regular program* is a program that does not contain assignment rules.

[3] In the syntax of the ASP solver `clingo`, and expression of the form $not\ (A_1, \dots, A_m)$ is written as `#false:` $A_1, \dots, A_m$ and it is a particular case of conditional literals.

Semantics. The semantics of programs with aggregates and partial functions is defined using a translation τ^* that turns a program into first-order sentences with equality over a signature σ_0 with three sorts *program*, *tuple*, and *set*.

We now describe the elements of the signature σ_0. We start by describing the predicate constants of σ_0. A predicate symbol is a pair p/k, where p is a symbolic constant and k is a nonnegative integer. About a rule or another syntactic expressions we say that it contains p/k if it contains an atom of the form $p(\mathbf{t})$ where $\mathbf{t}$ is a list of terms of length k. The set $\mathcal{P}_0$ of predicate constants of σ_0 consists of:

- all comparison symbols in (4) other than equality as binary predicate constants with arguments of sort program;
- all predicate symbols p/k occurring in the program with arguments of sort program; and
- the predicate constants $\in/2$ and $\notin/2$ with the first argument of sort tuple and the second argument of sort set;

We now turn to the function constants of σ_0. A *set symbol* is a pair $E/\mathbf{X}$ where E is an aggregate element and $\mathbf{X}$ is a list of lexicographically ordered variables. A variable is said to be global in aggregate atom if it occurs in its guard. A variable is global in a rule if it occurs in a non-negated basic literal or it is global in an aggregate atom. We say that set symbol $E/\mathbf{X}$ occurs in a rule R if E occurs in an aggregate atom in the body of R and $\mathbf{X}$ is the lexicographically ordered list of all global variables of R occurring in E. A set symbol occurs in a program if it occurs in some rule of the program. We now describe the set $\mathcal{F}_0$ of function symbols of σ_0, which is partitioned into four disjoint sets:

- *decidable functions* $\mathcal{F}_d$, which consists of $+\backslash 2$ with arguments and value of the sort program, and $count\backslash 1$ and $sum\backslash 1$ with argument of sort set and value of sort program;
- *rigid partial functions* $\mathcal{F}_r$, which contains $f\backslash k$ for every assigned function f occurring in an assignment of the form (6) in the program, where k is the length of $\mathbf{t}$; both arguments and value are of sort program;
- *set functions* $\mathcal{F}_s$, which consists of function constants of the form $s_{E/\mathbf{X}}\backslash k$ for each set symbol $E\backslash\mathbf{X}$ occurring in the program, where k is the length of $\mathbf{X}$; arguments are of the sort program and value of the set sort.
- *constructors functions* $\mathcal{F}_c$, which contains $f\backslash k$ for every program term $f(\mathbf{t})$ occurring in the program such that f is not in $\mathcal{F}_d \cup \mathcal{F}_r$, where k is the length of $\mathbf{t}$; both arguments and value are of sort program. It also contains object constants $inf\backslash 0$ and $sup\backslash 0$ with value of sort program; and $\overline{n}\backslash 0$ for every integer n; with value of sort program. In addition, it contains function constants $tuple\backslash k$ with arguments of the sort program and value of the tuple sort for each set aggregate element (5)..

We assume that rigid and set functions are intensional and that decidable and constructor functions are extensional. For predicates, only $\in/2$, $\notin/2$ and the comparison symbols in (4) are extensional; all other predicate symbols are intensional.

Note that every program term is also a term of the signature σ_0. Note also that precomputed terms only can contain occurrences of constructors and rigid functions. We say that a precomputed term t is *Herbrand* if it does not contain rigid function symbols.

Standard Interpretations. A *standard interpretation* I is an interpretation of σ_0 that satisfies:

- the universe of the sort program is the set containing all Herbrand terms;
- the universe of the sort tuple is the set containing all expressions of the form $tuple(d_1, \ldots, d_n)$ with each d_i an elements of the universe of the sort program;
- the universe of sort set is the set containing all subsets of the universe of the sort tuple;
- I interprets each Herbrand term as itself, and each term of the form $tuple(t_1, \ldots t_n)$ as the $tuple(t_1^I, \ldots t_n^I)$;
- I interprets predicate symbols $>, \geq, <, \leq, \neq$ according to the total order on precomputed terms selected above;
- $\in^I$ is the set of pairs (t, s) for which the t belongs to the set s, and $\notin^I$ is the set of pairs (t, s) for which the tuple t does not belong to s;
- $(t_1 + t_2)^I$ is $\overline{n_1 + n_2}$ if both t_1^I and t_2^I are numerals $\overline{n_1}$ and $\overline{n_2}$ respectively; and it is undefined otherwise; and
- for a term t of sort set, $sum(t)^I$ is $\widehat{\texttt{sum}}(t^I)$, and $count(t)^I$ is $\widehat{\texttt{count}}(t^I)$ where $\widehat{\texttt{sum}}(t^I)$ and $\widehat{\texttt{count}}(t^I)$ are defined as follows. $\widehat{\texttt{count}}(s)$ is the numeral corresponding to the cardinality of the set s, if s is finite; and sup otherwise. $\widehat{\texttt{sum}}(s)$ is the numeral corresponding to the sum of the weights of all tuples in s, if s contains finitely many tuples with non-zero weights; and 0 otherwise. If s is empty, then $\widehat{\texttt{sum}}(s) = 0$. The weight of $tuple(d_1, \ldots, d_n)$ is n if d_1 is a numeral $\overline{n}$, and it is 0 otherwise.[4]

Translation. The translation from logic programs to first-order sentences contains axioms that fix the intended meaning of rigid and set functions. By $RIG(\Pi)$ we denote the set of sentences of the form (2) for every function symbol f/n in $\mathcal{F}_r$. By $TOT(\Pi)$ we denote the set of sentences of the form (3) for every function symbol f/n in $\mathcal{F}_s$. By $SET(\Pi)$ we denote the set containing all the sentences in $TOT(\Pi)$ and all sentences of the form

$$\forall \mathbf{X} T \big(\ \ T \in s_{E/\mathbf{X}}(\mathbf{X}) \leftrightarrow ELM(T, \mathbf{X}) \big) \tag{8}$$

$$\forall \mathbf{X} T \big(\neg T \in s_{E/\mathbf{X}}(\mathbf{X}) \rightarrow T \notin s_{E/\mathbf{X}}(\mathbf{X}) \big) \tag{9}$$

for each function symbol $s_{E/\mathbf{X}}\backslash k$ in $\mathcal{F}_s$ with E is of the form (5), and where $ELM(T, \mathbf{X})$ is the formula

$$\exists \mathbf{Y} \left(T = tuple(t_1, \ldots, t_k) \wedge A_1 \wedge \cdots \wedge A_l \right),$$

$\mathbf{Y}$ is the list of all variables occurring in E that do not belong to $\mathbf{X}$. Both $\mathbf{X}$ and $\mathbf{Y}$ are lists of variables of the sort program. These sentences relate function symbols from $\mathcal{F}_s$ with their meaning in the program.

[4] We define the sum of an infinite set of non-zero weights to be 0 for compatibility with `Abstract Gringo` (Gebser et al. 2015). Given that our formalism allows for partial functions, a more natural choice would be to define this to be undefined. We prove latter that for programs without assignments, our semantics coincides with `Abstract Gringo`, so we follow the same convention.

Proposition 1. *If $\langle H, I\rangle$ is a standard HT-interpretation of σ_0 that satisfies (8-9), then for every tuple $\mathbf{g}$ of ground terms of sort program and every ground term t of the sort tuple, $I \models t \in s_{E/\mathbf{X}}(\mathbf{g})$ iff I satisfies:*

$$\exists \mathbf{Y}\,(t = tuple(t'_1, \ldots, t'_k) \wedge A'_1 \wedge \cdots \wedge A'_l) \qquad (10)$$

where E is an aggregate element of the form (5) and $t'_1, \ldots, t'_k$ and $A'_1, \ldots, A'_l$ are obtained from $t_1, \ldots, t_k$ and $A_1, \ldots, A_l$ by replacing variables in $\mathbf{X}$ with the corresponding term in $\mathbf{g}$. Furthermore, $H \models t \notin s_{E/\mathbf{X}}(\mathbf{g})$ iff $\langle H, I\rangle$ satisfies (10).

We now recursively define translation $\tau^*_{\mathbf{Z}}$, where $\mathbf{Z}$ is a list of variables, which is used to keep track of the global variables in the rule being translated:

1. for every atomic formula A occurring outside of an aggregate literal, its translation $\tau^*_{\mathbf{Z}}(A)$ is A; and for every literal of the form $not\ not\ A$, its translation is $\neg\neg A$;
2. for every literal of the form $not\ (A_1, \ldots, A_l)$, its translation is $\neg\exists \mathbf{Y}(A_1 \wedge \cdots \wedge A_l)$ where $\mathbf{Y}$ is the set of variables in $A_1, \ldots, A_l$ that do not occur in $\mathbf{Z}$; and
3. for an aggregate atom of the form $\#op\{E\} \prec u$, its translation $\tau^*_{\mathbf{Z}}$ is $op(s_{E/\mathbf{X}}(\mathbf{X})) \prec u$ where $\mathbf{X}$ is the lexicographically ordered list of variables in $\mathbf{Z}$ occurring in E.

For a rule R of the form (7) with global variables $\mathbf{Z}$:

1. if Hd is an atom of the form $p(\mathbf{t})$, then $\tau^*(R)$ is the universal closure of[5]

   $$\tau^*_{\mathbf{Z}}(B_1) \wedge \cdots \wedge \tau^*_{\mathbf{Z}}(B_n) \wedge \mathbf{t} = \mathbf{V} \rightarrow p(\mathbf{V});$$

   where $\mathbf{V}$ is a list of distinct fresh variables of the same length as $\mathbf{t}$;
2. if Hd is an assignment of the form $f(\mathbf{t}) := t'$, then $\tau^*(R)$ is the universal closure of

   $$\tau^*_{\mathbf{Z}}(B_1) \wedge \cdots \wedge \tau^*_{\mathbf{Z}}(B_n) \wedge \mathbf{t} = \mathbf{V} \wedge t' = Y \rightarrow f(\mathbf{V}) = Y$$

   where $\mathbf{V}$ is as in the previous item and Y is a fresh variable not occurring in $\mathbf{V}$;
3. if Hd is $\bot$, then $\tau^*(R)$ is the universal closure of

   $$\tau^*_{\mathbf{Z}}(B_1) \wedge \cdots \wedge \tau^*_{\mathbf{Z}}(B_n) \rightarrow \bot.$$

For a program Π, we define $\tau^*(\Pi)$ as the set of sentences

$$RIG(\Pi) \cup SET(\Pi) \cup \{\, \tau^*(R) \mid R \in \Pi \,\}.$$

We say that I is a *stable model* of a program Π if I is a standard stable model of $\tau^*(\Pi)$. If I is a standard interpretation, by $I^{\downarrow}$ we denote the set of atomic formulas of the forms $p(\mathbf{t})$ and $f(\mathbf{t}) = t'$ satisfied by I such that p/k and $f\backslash k$ respectively are predicate and rigid partial function symbols occurring in the program, $\mathbf{t}$ is a tuple of Herbrand terms and t' is a Herbrand term. If I is a stable model of Π, we say that $I^{\downarrow}$ is an *answer set* of Π.

Theorem 2. *The answer sets of a regular program Π are exactly its stable models as defined by* `Abstract Gringo` *(Gebser et al. 2015).*

[5] If $\mathbf{t_1}$ and $\mathbf{t_2}$ are two lists $t_{11}, \ldots, t_{1k}$ and $t_{21}, \ldots, t_{2k}$ of terms of the same length, then $\mathbf{t_1} = \mathbf{t_2}$ is an abbreviation for the conjunction: $(t_{11} = t_{21}) \wedge \ldots \wedge (t_{1k} = t_{2k})$.

4.1 Choice Rules and Aggregate Assignments

Choice rules are a convenient way to express non-deterministic behavior in logic programs. As an example, we can describe the coloring problem as follows:

```
ass(V) := #some{C: color(C)} :- vertex(V).   (11)
:- edge(V1, V2), ass(V1) = ass(V2).          (12)
```

where (11) is a choice rule describing all possible assignments of colors to nodes in a graph.

In our language, we allow two types of choice rules: *some-choice rules* such as (11) allow to express most of the common patterns of non-deterministic assignments while keeping the assigned function to the left, while *braced-choice rules* introduced next allow a finer control using a syntax that is closer to the one of regular ASP. A *braced-choice rule* is an expression of the form

$$l\,\{Hd : L_1, \ldots, L_m\}\,u \text{ :- } B_1, \ldots, B_n, \qquad (13)$$

where Hd is an atom or an assignment, and l and u are non-negative integers with $l \leq u$ called *lower* and *upper bounds*, respectively. If m is 0, we omit the colon and the empty list. A choice rule of the form (13) is understood as an abbreviations for the rules

$$Hd \text{ :- } B_1, \ldots, B_n,\ not\ not\ Hd',\ L_1, \ldots, L_m,$$
$$\bot \text{ :- } B_1, \ldots, B_n,\ \#\mathtt{count}\{Hd'' : Hd', L_1, \ldots, L_m\} < l,$$
$$\bot \text{ :- } B_1, \ldots, B_n,\ \#\mathtt{count}\{Hd'' : Hd', L_1, \ldots, L_m\} > u.$$

where Hd' and Hd'' are identical to Hd if Hd is an atom. Otherwise, Hd is an assignment of the form $f(t_1, \ldots, t_k) := t_0$, and Hd' is the atom obtained from Hd by replacing the assignment operator $:=$ by the equal symbol $=$ in Hd, and Hd'' is the term $\hat{f}(t_1, \ldots, t_k, t_0)$ where $\hat{f}$ is a fresh function constant. Both or any of the bounds can be omitted. In that case, the corresponding constraint is omitted from the translation.

As an example, we can use the following braced-choice rule to replace (11):

```
1{ass(V) :=C: color(C)} :- vertex(V).   (14)
```

This rule states that *ass*$\backslash 1$ is a function that assigns to each vertex V a color C. The lower bound of 1 states that *ass*$\backslash 1$ is a total function. If omitted, *ass*$\backslash 1$ would be a partial function and we would have stable models where some vertices are not assigned any color.

A *some-choice rule* is an expression of the form

$$f(\mathbf{t})\ :=\ \#\mathtt{some}\{t_1, \ldots, t_k : L_1, \ldots, L_m\} \text{ :- } B_1, \ldots, B_n,$$

and it is understood as an abbreviation for the braced-choice rule

$$1\{f(\mathbf{t}) := t_1 : L_1, \ldots, L_m\} \text{ :- } B_1, \ldots, B_n,$$
$$\#\mathtt{count}\{t_1, \ldots, t_k : L_1, \ldots, L_m\} > 0.$$

Rules (11) and (14) can be used interchangeably in the context of the coloring problem because `color`$\backslash 1$ has a non-empty extension. However, in general, these two rules have different semantics. If `color`$\backslash 1$ would have an empty-extension, then rule (14) would be unsatisfiable and we would have no stable

```
posX(I) := #some{X: X=0..W-size(I)} :- area(W,H).
posY(I) := #some{Y: Y=0..H-size(I)} :- area(W,H).
:- I1!=I2, posX(I2)-posX(I1)=0..size(I1)-1, posY(I2)-posY(I1)=0..size(I1)-1.
:- I1!=I2, posX(I2)-posX(I1)=0..size(I1)-1, posY(I2)-posY(I1)+size(I2)=1..size(I1).
```

Figure 1: Program representing the Packing Problem using some-choice rules

model, while rule (11) would be satisfied and leave `ass(V)` undefined.

As another example, let us consider the Packing Problem (Calimeri et al. 2011): *Given a rectangular area of a known dimension and a set of squares, each of which has a known dimension, the problem is to pack all the squares into the rectangular area such that no two squares overlap each other.* We can represent this problem by the program in Figure 1, where `area` encodes the dimensions of the rectangular area, `size` the size of each square and `posX` and `posY` the position of each square in the x and y coordinate respectively. For comparison, the rule in line 3 uses 2 variables, `I1` and `I2`, to denote the identifiers of the squares. The regular ASP encoding of this same rule (Calimeri et al. 2011) uses eight more variables:

```
:- I1 != I2, size(I1,D1), size(I2,D2),
   pos(I1,X1,Y1), pos(I2,X2,Y2),
   W1=X1+D1, H1=Y1+D1,
   X2>=X1, X2<W1, Y2>=Y1, Y2<H1.
```

5 Translating into regular logic programs

Cabalar (2011) showed how logic programs with rigid functions and without aggregates can be translated into regular logic programs. We show here that this translation can be extended to logic programs with aggregates. This process can be divided in two steps. First, we rewrite the logic program into a *plain* logic program. A logic program is called *plain*[6] if every occurrence of a rigid partial function symbol $f\backslash k$ in the program is either in assignment of the form $f(\mathbf{t}) := t'$ or in an comparison of the form $f(\mathbf{t}) = t'$ where $\mathbf{t}$ is a list of terms and t' is a term such that neither contains any rigid function symbol. Then, a plain program is *flatten* (Naish 1991; Rouveirol 1994) into a regular logic program by replacing every rigid function symbol $f\backslash k$ by a fresh predicate symbol $p_f/(k+1)$ and adding uniqueness constraints.

Any logic program can be transformed into an equivalent plain logic program. The following general about rigid functions is useful for proving this equivalence.

Proposition 2. *Let $A(X)$ be an atomic formula of which X is subterm, t be a term that does not contain X and G be the conjunction of formula* (2) *for every function symbols $f\backslash k$ that occur in in t. Then, sentence*[7]*:*

$$G \to \widetilde{\forall}(A(t)) \leftrightarrow \exists X(A(X) \wedge t = X)) \qquad (15)$$

is satisfied by all HT-interpretations.

We can use Proposition 2 to turn any logic program into an equivalent plain logic program as follows.

[6]The notion of plain is an adaption of a similar notion defined by Bartholomew and Lee (2019) to allow assignments.

[7]For a formula F, by $\widetilde{\forall}F$ we denote the universal closure of F.

An atomic formula of the form $t = t'$ is called an *equation*. An occurrence of a term of the form $f(\mathbf{t})$ is *rough* if $f\backslash k$ is a rigid function symbol and the term is not the left-hand side of an assignment of or an equation. A logic program is plain iff it does not contain any rough occurrences. Hence, we transform a logic program into an equivalent plain logic program by repeatedly removing one rough occurrence at a time. We remove a rough occurrence $f(\mathbf{t})$ by replacing that occurrence by a fresh variable X, and adding the equation $f(\mathbf{t}) = X$. Where the equation is added depends on the context of the rough occurrence and it is defined as follows:

- if the rough occurrence is in the head of a rule, in an atomic formula in the body of a rule or in the guard of an aggregate atom, then we add the equation in the body of the rule;
- if the rough occurrence is in a literal of the form $not\ not\ A$, then we add the equation to the body but preceded it by double negation $not\ not\ f(\mathbf{t}) = X$;
- if the rough occurrence is in a literal of the form $not\ (A_1, \ldots, A_l)$, then we add the equation to the list of negated literals, that is, we replace the literal by $not\ (A'_1, \ldots, A'_l, f(\mathbf{t}) = X)$ where is A'_i either A_i or the result of replacing the rough occurrence by X in A_i;
- if the rough occurrence is in an aggregate element, then we add the equation to the list of conditions of that aggregate element.

By $plain(\Pi)$ we denote the plain logic program obtained from Π by repeatedly removing rough occurrences of rigid function symbols until there is no more rough occurrences. At each step, the number of rough occurrences is reduced by one, so the process terminates after a finite number of steps.

Theorem 3. *The answer sets of programs Π and $plain(\Pi)$ coincide.*

Theorem 3 shows that plain logic programs constitute a normal form for logic programs.

Let us now show how to flatten a plain logic program. Given a plain logic program Π, by $flat(\Pi)$ we denote the regular logic program obtained from Π by replacing every assignment of the form $f(\mathbf{t}) := t'$ and every equation of the form $f(\mathbf{t}) = t'$ with $f\backslash k$ a rigid function symbol by an atom of the form $p_f(\mathbf{t}, t')$ where $p_f/(k+1)$ is a fresh predicate symbol, and adding the following uniqueness constraint for each rigid function symbol $f\backslash k$ that occurs in Π:

$$\bot \text{ :- } p_f(\mathbf{X}, Y),\ \texttt{\#count}\{Z : p_f(\mathbf{X}, Z)\} > 1. \qquad (16)$$

where $\mathbf{X}$ is a list of distinct variables of length k and Y and Z are variables that do not occur in $\mathbf{X}$. For non-plain programs, we define $flat(\Pi)$ as $flat(plain(\Pi))$.

We now describe the correspondence between the answer sets of a logic program Π and its flattening $flat(\Pi)$. If $\mathcal{I}$ is

	funasp	clingof	ASPMT	fasp
Variables	✓	✓	✓	
Nonmonot. func	✓	✓	✓	
Non-tight progr.	✓	✓		✓
Intel. Grounding	✓	✓		
Partial functions	✓	✓		
Aggregates	✓	✓		
Nested functions	✓			
Arbitrary values	✓			
Backend	clingo	—	z3	csp

Table 1: Comparison of funasp with other systems that support non-Herbrand functions. funasp, ASPMT, and fasp are implemented by translating into different backends, while clingof is implemented as a fork of clingo version 3.

an answer set of Π, by $\mathit{flat}(\mathcal{I})$ we denote the set obtained from $\mathcal{I}$ by replacing every atom of the form $f(\mathbf{t}) = t'$ by an atom of the form $p_f(\mathbf{t}, t')$ where $p_f/(k+1)$ is the predicate symbol corresponding to function symbol $f\backslash k$.

Theorem 4. *The mapping $\mathcal{I} \mapsto \mathit{flat}(\mathcal{I})$ is a one-to-one correspondence between the answer sets of programs Π and $\mathit{flat}(\Pi)$.*

6 Implementation and Evaluation

Implementation. We implement a solver, which we call funasp, which extends the ASP solver clingo (Gebser et al. 2019) to support intensional functions as described in Section 4. In addition, to the language described above, funasp supports all the constructs of clingo, and intensional functions can be used in any of those constructs, including other types of aggregates, optimization statements, etc. Our implementation is built on top of version 6 of the ASP solver clingo[8] using its Python API. The implementation first walks the abstract syntax tree of the input program rewriting the abbreviations described in Section 4.1 and recoding the rigid partial functions. Then, it applies the translation described in Section 5 to obtain a regular logic program, which is then passed to clingo for solving. The output of clingo is then post-processed to obtain the answer sets of the original program. The implementation is submitted in the Supplementary Material and will be made publicly available under the MIT License after the review process. There are three systems that are based on extensions of ASP with non-Herbrand functions: fasp (Lin and Wang 2008), clingof (Balduccini 2013), and ASPMT (Bartholomew and Lee 2013). Though, the theoretical underpinnings of these systems are are closely related to our approach, as shown in Section 3, the systems themselves target different goals. These three systems focus on using non-Herbrand functions to represent large numerical domains and exploit the functional nature of the representations to improve the performance of the solver, similar to Constraint ASP (Lierler 2014). However, none of these systems support the use of nested functions nor allow their values to be arbitrary terms.

[8]Available at https://github.com/potassco/clingo/tree/wip-20.

ASPMT and fasp have some extra limitations, such as not supporting intelligent grounding, aggregates or partial functions. fasp does not support the use of variables and ASPMT only support tight programs. Table 1 summarizes the main features of these systems.

Experimental evaluation. Since the systems based on non-Herbrand functions do not support the same features, they cannot represent the same problems. Thus, to evaluate the performance of funasp, we compare it with clingo on a set of benchmarks from the ASP competitions (Calimeri, Ianni, and Ricca 2014; Alviano et al. 2013) using the original encodings for clingo and rewriting those encodings to use intensional functions for funasp. We select nine problems that have predicates that represent functions and we run 20 instances for each problem. The following Table reports the average time taken by funasp and clingo to solve these instances, as well as the number of instances that timeouts.

	clingo time.timeout	funasp time	timeout.
Disj. Scheduling	562.50.15	449.32	11.
Graceful Graphs	340.53. 9	355.85	10.
Graph Colouring	445.17.13	441.77	13.
Incr. Scheduling	548.62.19	548.29	19.
Ricochet Robots	0.04. 0	0.08	0.
Solitaire	149.00. 3	109.66	3.
Stable-Marriage	35.09. 0	40.12	0.
Visit-all	367.95. 9	333.72	8.
WeightedSeq.	184.01. 4	61.47	0.
Total	323.29.92	294.03	84.

We set a timeout of 600s and ran our experiments on a machine with an Intel Core i7-7700T CPU@2.90GHz and 32GB of RAM. We can see that funasp solves 8 more instances and on average is 10% faster than clingo on these benchmarks. This may seem surprising at first, since funasp uses clingo for solving. This means that clingo is faster when using the encodings produced by funasp than when using the original encodings. This improvement seems due to the inclusion of explicit uniqueness constraints (16).

7 Conclusions

We have introduced a free version of the Quantified Here-and-There logic, shown that generalizes several existing approaches to ASP with non-Herbrand functions, and that also covers aggregates. We have used this framework to define an extension of ASP where user can define their own functions, and shown that this is a conservative extension of the semantics of Abstract Gringo. We also have shown how to translate this new language into standard ASP, and implemented a new system called funasp that extends clingo to support this new language, showing that the increase in expressivity does not come with a performance penalty. For future work, we plan to explore how the functional representation can be exploited to improve the performance of ASP solvers for large domains base on the lessons from the works by Lin and Wang (2008); Balduccini (2013); Bartholomew and Lee (2013), but without compromising the expressivity of our language.

References

Alviano, M.; Calimeri, F.; Charwat, G.; Dao-Tran, M.; Dodaro, C.; Ianni, G.; Krennwallner, T.; Kronegger, M.; Oetsch, J.; Pfandler, A.; Pührer, J.; Redl, C.; Ricca, F.; Schneider, P.; Schwengerer, M.; Spendier, L.; Wallner, J.; and Xiao, G. 2013. The Fourth Answer Set Programming Competition: Preliminary Report. In Cabalar, P.; and Son, T., eds., *Proceedings of the Twelfth International Conference on Logic Programming and Nonmonotonic Reasoning (LPNMR'13)*, volume 8148 of *Lecture Notes in Artificial Intelligence*, 42–53. Springer-Verlag.

Balduccini, M. 2013. ASP with non-herbrand partial functions: A language and system for practical use. *Theory and Practice of Logic Programming*, 13(4-5): 547–561.

Bartholomew, M.; and Lee, J. 2013. Functional Stable Model Semantics and Answer Set Programming Modulo Theories. In (Rossi 2013), 718–724.

Bartholomew, M.; and Lee, J. 2014. Stable Models of Multi-Valued Formulas: Partial vs. Total Functions. In Baral, C.; De Giacomo, G.; and Eiter, T., eds., *Proceedings of the Fourteenth International Conference on Principles of Knowledge Representation and Reasoning (KR'14)*, 583–586. AAAI Press.

Bartholomew, M.; and Lee, J. 2019. First-order stable model semantics with intensional functions. *Artificial Intelligence*, 273: 56–93.

Bencivenga, E. 2002. Free Logics. In Gabbay, D.; and Guenthner, F., eds., *Handbook of Philosophical Logic*, 147–196. Springer-Verlag.

Cabalar, P. 2011. Functional answer set programming. *Theory and Practice of Logic Programming*, 11(2-3): 203–233.

Cabalar, P.; Fariñas del Cerro, L.; Pearce, D.; and Valverde, A. 2014. A Free Logic for Stable Models with Partial Intensional Functions. In Fermé, E.; and Leite, J., eds., *Proceedings of the Fourteenth European Conference on Logics in Artificial Intelligence (JELIA'14)*, volume 8761 of *Lecture Notes in Artificial Intelligence*, 340–354. Springer-Verlag.

Calimeri, F.; Ianni, G.; and Ricca, F. 2014. The Third Open Answer Set Programming Competition. *Theory and Practice of Logic Programming*, 14(1): 117–135.

Calimeri, F.; Ianni, G.; Ricca, F.; Alviano, M.; Bria, A.; Catalano, G.; Cozza, S.; Faber, W.; Febbraro, O.; Leone, N.; Manna, M.; Martello, A.; Panetta, C.; Perri, S.; Reale, K.; Santoro, M.; Sirianni, M.; Terracina, G.; and Veltri, P. 2011. The Third Answer Set Programming Competition: Preliminary Report of the System Competition Track. In Delgrande, J.; and Faber, W., eds., *Proceedings of the Eleventh International Conference on Logic Programming and Nonmonotonic Reasoning (LPNMR'11)*, volume 6645 of *Lecture Notes in Artificial Intelligence*, 388–403. Springer-Verlag.

Fandinno, J.; and Lifschitz, V. 2023. Omega-Completeness of the Logic of Here-and-There and Strong Equivalence of Logic Programs. In Marquis, P.; Son, T.; and Kern-Isberner, G., eds., *Proceedings of the Twentieth International Conference on Principles of Knowledge Representation and Reasoning (KR'23)*, 240–251.

Fariñas del Cerro, L.; Pearce, D.; and Valverde, A. 2013. FQHT: The Logic of Stable Models for Logic Programs with Intensional Functions. In (Rossi 2013), 891–897.

Gebser, M.; Harrison, A.; Kaminski, R.; Lifschitz, V.; and Schaub, T. 2015. Abstract Gringo. *Theory and Practice of Logic Programming*, 15(4-5): 449–463.

Gebser, M.; Kaminski, R.; Kaufmann, B.; and Schaub, T. 2019. Multi-shot ASP solving with clingo. *Theory and Practice of Logic Programming*, 19(1): 27–82.

Lierler, Y. 2014. Relating constraint answer set programming languages and algorithms. *Artificial Intelligence*, 207: 1–22.

Lifschitz, V. 2012. Logic Programs with Intensional Functions. In Brewka, G.; Eiter, T.; and McIlraith, S., eds., *Proceedings of the Thirteenth International Conference on Principles of Knowledge Representation and Reasoning (KR'12)*. AAAI Press.

Lifschitz, V. 2019. *Answer Set Programming*. Springer-Verlag.

Lin, F.; and Wang, Y. 2008. Answer Set Programming with Functions. In Brewka, G.; and Lang, J., eds., *Proceedings of the Eleventh International Conference on Principles of Knowledge Representation and Reasoning (KR'08)*, 454–465. AAAI Press.

Naish, L. 1991. Adding equations to NU-Prolog. In Maluszyński, J.; and Wirsing, M., eds., *Programming Language Implementation and Logic Programming*, 15–26. Berlin, Heidelberg: Springer-Verlag.

Pearce, D. 2006. Equilibrium logic. *Annals of Mathematics and Artificial Intelligence*, 47(1-2): 3–41.

Pearce, D.; and Valverde, A. 2008. Quantified Equilibrium Logic and Foundations for Answer Set Programs. In Garcia de la Banda, M.; and Pontelli, E., eds., *Proceedings of the Twenty-fourth International Conference on Logic Programming (ICLP'08)*, volume 5366 of *Lecture Notes in Computer Science*, 546–560. Springer-Verlag.

Rossi, F., ed. 2013. *Proceedings of the Twenty-third International Joint Conference on Artificial Intelligence (IJCAI'13)*. IJCAI/AAAI Press.

Rouveirol, C. 1994. Flattening and Saturation: Two Representation Changes for Generalization. *Machine Learning*, 14(2): 210–232.